Department of Information Technology and Management
Faculty of Science and Information Technology

Academic Project Report on

SkillBridge AI

An AI-Based Job and Skill Recommendation Mobile Application

Course Code: ITM 314

Course Title: Mobile Application Development (Android) Lab

Semester: Spring 2026

Submitted By

Name	ID
Kazi Arafat Hossain	232-51-008
Md. Aftab Uddin Ahmed	232-51-017
Md. Iftimam Montasir Nahid	232-51-039

Submitted To

Fahad Mahmud
Lecturer, Department of ITM
Daffodil International University

Date of Submission: 24th April 2026

Abstract

SkillBridge AI is a mobile application built using the Flutter framework and Dart programming language. It is designed to help job seekers in Bangladesh, especially fresh graduates and young professionals, find the right jobs and courses based on their skills. The app is aligned with the United Nations Sustainable Development Goal 8 (SDG-8), which focuses on decent work and economic growth.

The application uses a hybrid recommendation engine that combines two models. The Primary Model uses TF-IDF, Cosine Similarity, and Jaccard Coefficient, achieving 82.3% test accuracy. The Improved Model uses Sentence-BERT (the all-MiniLM-L6-v2 transformer) with Cosine Similarity and Jaccard Coefficient, achieving 85.0% test accuracy. Both models were trained and evaluated in Google Colab on a dataset of 50,000 job postings across 7 industries, and all model files were exported as .pkl files for use in the mobile app backend. The backend uses Firebase for user authentication and data storage.

Key features of the app include a skill input screen, job matching results, skill gap analysis, course recommendations, a career guidance chatbot, a confidence tracker, geographic job insights, and an application tracker. The app supports both dark and light themes, and is built with clean, maintainable code following Flutter best practices.

This report documents the complete development process of SkillBridge AI, from the initial problem statement and design decisions to the implementation, testing, and future plans.

Table of Contents

Chapter 1: Introduction

1.1 Introduction

SkillBridge AI is a mobile application built as a semester project for the Mobile Application Development Lab course. The app is designed to solve a very real problem: many young people in Bangladesh struggle to find jobs that match their skills, and they do not always know which skills they need to improve. SkillBridge AI acts as a personal career assistant that helps users discover jobs, find their skill gaps, and get course recommendations, all in one place.

The app is built using Flutter, which means it works on both Android and iOS devices from a single codebase. It connects to Firebase for user login and data storage, and uses Google Gemini AI to provide a helpful chatbot for career advice. The name 'SkillBridge AI' reflects the app's core purpose: to bridge the gap between the skills a user has and the skills the job market needs.

The project is also aligned with the United Nations SDG-8 (Decent Work and Economic Growth), which makes it meaningful beyond just a class assignment. It aims to make career guidance accessible to everyone, not just those who can afford expensive consultants.

1.2 Motivation

The main motivation for building SkillBridge AI came from observing how difficult it is for students and fresh graduates in Bangladesh to navigate the job market. There are many job portals available, but most of them simply list jobs without helping the user understand if they are a good fit. Users have to manually search through hundreds of listings and often apply to jobs they are not qualified for, wasting both their time and the employer's time.

Additionally, many users do not know what skills are in demand in their industry, or which courses they should take to improve their chances. SkillBridge AI was motivated by the desire to make this process smarter, faster, and more personalized using artificial intelligence and machine learning techniques.

1.3 Objectives

The main objectives of the SkillBridge AI project are as follows:

1. To build a fully functional cross-platform mobile application for career guidance and job matching using Flutter and Dart.
2. To implement a smart job recommendation engine that uses SBERT, TF-IDF, Cosine Similarity, Jaccard Coefficient, and other ML techniques to match users to suitable jobs.
3. To provide a skill gap analysis feature that tells users exactly which skills they are missing for a target job.
4. To offer personalized course recommendations that help users close their skill gaps.
5. To integrate Google Gemini AI into a chatbot that can answer career-related questions in a conversational way.
6. To implement a secure user authentication system using Firebase, allowing users to register and log in safely.
7. To support dark and light themes for a better user experience.

8. To align the application with UN SDG-8 to promote decent work and economic growth in Bangladesh.

1.4 Expected Outcomes

After completing this project, the following outcomes were expected:

- A working Android application (APK) built with Flutter that users can install and use on their phones.
- A smart ML-powered recommendation engine that provides accurate job and course suggestions based on user skills.
- A clear and well-organized source code following Flutter and Dart best practices, with good separation of concerns (screens, services, ML logic, data, models).
- A practical tool that genuinely helps job seekers in Bangladesh make better career decisions.
- A detailed project report documenting every step of the development process.

Chapter 2: Background

2.1 Preliminaries / Terminologies

This section explains the key terms and technologies used in the SkillBridge AI project.

- **Flutter & Dart:** Flutter is a UI framework made by Google. It lets developers write one codebase and deploy the app on Android, iOS, and Web. Dart is the programming language used with Flutter. Flutter's hot reload feature makes development very fast.
- **Firebase:** A backend service from Google. SkillBridge AI uses Firebase Authentication for user login and registration, and Firestore (a NoSQL cloud database) for storing user data and career profiles.
- **TF-IDF (Term Frequency-Inverse Document Frequency):** A method from information retrieval used to measure how important a word (or skill) is in a document compared to a collection of documents. In SkillBridge AI, it is used to score how well a user's skills match a job description.
- **Cosine Similarity:** A mathematical method used to compare two vectors (lists of numbers) by measuring the angle between them. It is used in SkillBridge AI to compare a user's skill vector to a job's skill vector.
- **Jaccard Coefficient:** A simple method to measure how similar two sets are, based on how many items they share. Used to boost recommendation scores for jobs that share many skills with the user.
- **Sentence-BERT (SBERT):** A special version of the BERT transformer model, designed to produce sentence-level embeddings. In SkillBridge AI, the all-MiniLM-L6-v2 model is used. It converts any text (like a user's skills or a job description) into a list of 384 numbers (called a vector) that captures the meaning of the text, not just the keywords. This means it can understand that "talent acquisition" and "recruitment" are related, even if the exact words are different. SBERT is the foundation of the Improved Recommendation Model.
- **Proxy Evaluation:** Because the dataset has no real user-job interaction labels (like "this user applied for this job"), a proxy method is used to measure model accuracy. Each test job is queried using only its first 2 required skills, and the model must

predict the correct Industry. This gives a meaningful measure of how well the model understands skill-to-industry relationships.

- **PKL Files (.pkl):** Python pickle files are used to save trained ML models to disk. After training in Google Colab, the TF-IDF vectorizer, SBERT job embeddings, label encoder, and job data are all saved as .pkl files. These files are then loaded by the Flask/FastAPI backend server that the mobile app calls for recommendations.
- **Gemini AI:** Google's large language model (AI). It is integrated via an API to power the chatbot (Career Guidance Assistant) in the app, providing personalized advice based on user input.
- **Provider:** A state management package for Flutter. SkillBridge AI uses the Provider pattern to share app state (like the user's career profile) across all screens without passing data manually.
- **SDG-8:** Sustainable Development Goal 8 by the United Nations, which focuses on promoting inclusive and sustainable economic growth, employment, and decent work for all.
- **Skill Gap:** The difference between the skills a user currently has and the skills required for a target job. SkillBridge AI identifies these missing skills and recommends courses to fill them.

2.2 Related Works

Many applications and research papers have addressed the problem of job recommendation and career guidance. Popular platforms like LinkedIn, Bdjobs, and Glassdoor provide job listings and some level of matching, but they typically do not offer deep skill gap analysis or personalized course recommendations. Research in the field of intelligent tutoring systems and recommender systems has explored techniques like collaborative filtering, content-based filtering, and hybrid approaches.

SkillBridge AI is most closely related to Learning-Based Job Recommender Systems (JRS), as described in the academic literature. These systems combine user skill profiles with job requirement data and apply similarity algorithms to rank job matches. The app also draws inspiration from career readiness scoring frameworks that measure how prepared a user is for a specific role.

Unlike general job portals, SkillBridge AI is specifically designed for the Bangladeshi market, with job data and categories relevant to local industries like Software, Finance, Healthcare, Marketing, Manufacturing, Retail, and Education.

2.3 Comparative Analysis

The table below compares SkillBridge AI with other common tools:

Feature	Bdjobs	LinkedIn	Generic Job App	SkillBridge AI
Skill-Based Matching	No	Partial	No	Yes (2 ML models)
Skill Gap Analysis	No	No	No	Yes
Course Recommendations	No	Yes	No	Yes
AI Chatbot	No	No	No	Yes (Gemini AI)
Semantic Understanding	No	No	No	Yes (Sentence-BERT)
Local Market Focus	Yes	No	No	Yes
Dark/Light Theme	No	Yes	Partial	Yes
SDG-8 Aligned	No	No	No	Yes

The key advantage of SkillBridge AI over existing tools is its dual-model architecture. The TF-IDF model is fast and easy to understand, while the Sentence-BERT model goes deeper by understanding the meaning of skills, not just matching keywords. No existing Bangladeshi job platform offers this level of personalized, AI-driven career guidance in a free mobile app.

2.4 Challenges

Several challenges were encountered during the development of SkillBridge AI:

- **ML Integration in Flutter:** Implementing TF-IDF, Cosine Similarity, and other ML algorithms directly in Dart (Flutter's language), without using Python or external ML servers, required careful design and optimization.
- **Firebase & Provider Scoping:** Screens pushed using Navigator.push() sometimes ended up outside the Provider tree, causing blank grey screens. This was fixed by wrapping destination screens in ChangeNotifierProvider.value() before navigating.
- **Gemini API Integration:** Wiring the Gemini AI chatbot required creating a REST API client in Dart, handling streaming responses, and securely injecting the API key at compile time using --dart-define flags.

- **Navigation Architecture:** With more than 20 screens, managing the navigation stack and ensuring each screen could access the correct state required a well-structured routing system.
- **Dark/Light Theme Persistence:** Ensuring the theme choice was saved and correctly loaded on app restart required proper use of SharedPreferences and the Provider pattern.

Chapter 3: Requirement Specification

3.1 Tools and Technologies Required

The following tools and technologies were used to build SkillBridge AI:

- **Programming Language:** Dart (SDK ^3.8.1)
- **Framework:** Flutter (cross-platform, supports Android, iOS, and Web)
- **IDE:** Android Studio or Visual Studio Code with Flutter and Dart extensions
- **Backend:** Firebase (Firebase Authentication + Cloud Firestore)
- **AI Service:** Google Gemini API integrated via the gemini_chat_service.dart service
- **State Management:** Provider package (ChangeNotifier pattern)
- **Local Storage:** SharedPreferences for theme and learner preference persistence
- **Version Control:** Git and GitHub
- **Key Packages:** provider, firebase_core, firebase_auth, cloud_firestore, shared_preferences, flutter_animate
- **sentence-transformers:** A Python library used in Google Colab to run the Sentence-BERT model (all-MiniLM-L6-v2) for generating semantic job embeddings.
- **scikit-learn:** Used for TF-IDF vectorization, Logistic Regression (for evaluation), Nearest Neighbors, and all evaluation metrics.
- **Google Colab:** The cloud notebook environment where both ML models were trained and evaluated on the 50,000-job dataset. All model artifacts were saved to Google Drive as .pkl files.
- **pickle (.pkl files):** The trained models (TF-IDF vectorizer, SBERT embeddings, label encoder, job data, config) are exported as .pkl files for use by the mobile app's backend API.

3.2 Requirement Collection and Analysis

The requirements for SkillBridge AI were gathered by analyzing the challenges faced by young job seekers in Bangladesh and studying existing career guidance platforms. Requirements are split into two categories: functional (what the app must do) and non-functional (how the app must perform).

Functional Requirements

1. **User Authentication:** The app must allow users to register and log in securely using Firebase Authentication. It must support email/password login and Google Sign-In.
2. **Profile Input:** Users must be able to enter their career profile, including their current skills, education, experience, and preferred industry.
3. **Job Matching:** The system must recommend relevant jobs based on the user's skills using TF-IDF, Cosine Similarity, Jaccard Coefficient, and other ML algorithms.

4. **Skill Gap Analysis:** The app must show users which skills they are missing for a target job, along with a match score.
5. **Course Recommendations:** The app must suggest learning resources and courses that help fill the user's skill gaps.
6. **AI Career Chatbot:** The app must include a chatbot powered by Google Gemini AI that can answer career questions in a conversational way.
7. **Confidence Tracker:** Users must be able to rate and track their confidence across different career-related categories over time.
8. **Application Tracker:** Users must be able to track the status of their job applications (e.g., Applied, Interview, Rejected, Offer).
9. **Geographic Insights:** The app must show job market data by location to help users understand regional demand.
10. **Workforce Insights:** The app must display trends in skills demand and workforce changes across different industries.
11. **Assessment Hub:** The app must provide skill assessments to help users gauge their readiness for different roles.
12. **Logout:** Users must be able to log out from both the Home and Dashboard screens.

Non-Functional Requirements

1. **Performance:** The ML recommendation engine must return results quickly on a mobile device.
2. **Usability:** The interface must be clean, simple, and intuitive for users who may not be tech-savvy.
3. **Security:** User credentials must be handled securely by Firebase. Sensitive data like API keys must be injected at build time, not hardcoded.
4. **Theme Support:** The app must support both dark and light themes, with the choice persisted across sessions.
5. **Scalability:** The architecture must allow new screens and features to be added easily without breaking existing functionality.
6. **Code Quality:** The codebase must follow Flutter best practices, with separate layers for screens, services, ML logic, data, and models.

3.3 Use Case Modeling and Description

The main actors in the SkillBridge AI system are the Job Seeker (User) and the AI System (backend/Gemini).

Use Case Descriptions: Job Seeker

1. **Register / Login:** The user creates an account or logs in using their email and password or Google account. Firebase handles authentication.
2. **Enter Career Profile:** The user fills in their skills, education level, years of experience, and target industry.
3. **View Job Matches:** The system uses the user's profile to run the ML recommendation engine and display a ranked list of matching jobs.
4. **View Skill Gap:** The user selects a job from the results and sees which skills they have and which skills they are missing for that role.
5. **Browse Courses:** The app recommends courses to fill skill gaps. The user can browse and filter these recommendations.
6. **Use Chatbot:** The user can open the AI chatbot to ask career questions and receive personalized advice from Google Gemini AI.

7. **Track Applications:** The user can add job applications and track their progress through different stages.
8. **Track Confidence:** The user rates their confidence in different skill areas, and the app tracks these over time.
9. **View Insights:** The user can explore geographic and workforce insights to understand where jobs are and what skills are in demand.
10. **Manage Profile & Settings:** The user can update their profile, change the app theme, and manage privacy settings.
11. **Logout:** The user can safely log out from the Home or Dashboard screen.

Chapter 4: Design Specification

4.1 System Architecture

SkillBridge AI follows a clean layered architecture. The front end is built entirely with Flutter and Dart. The app uses the Provider pattern for state management, with AppState as the central ChangeNotifier that holds the user's career profile, theme preference, learner preferences, and authentication status.

The backend is a serverless setup powered by Firebase:

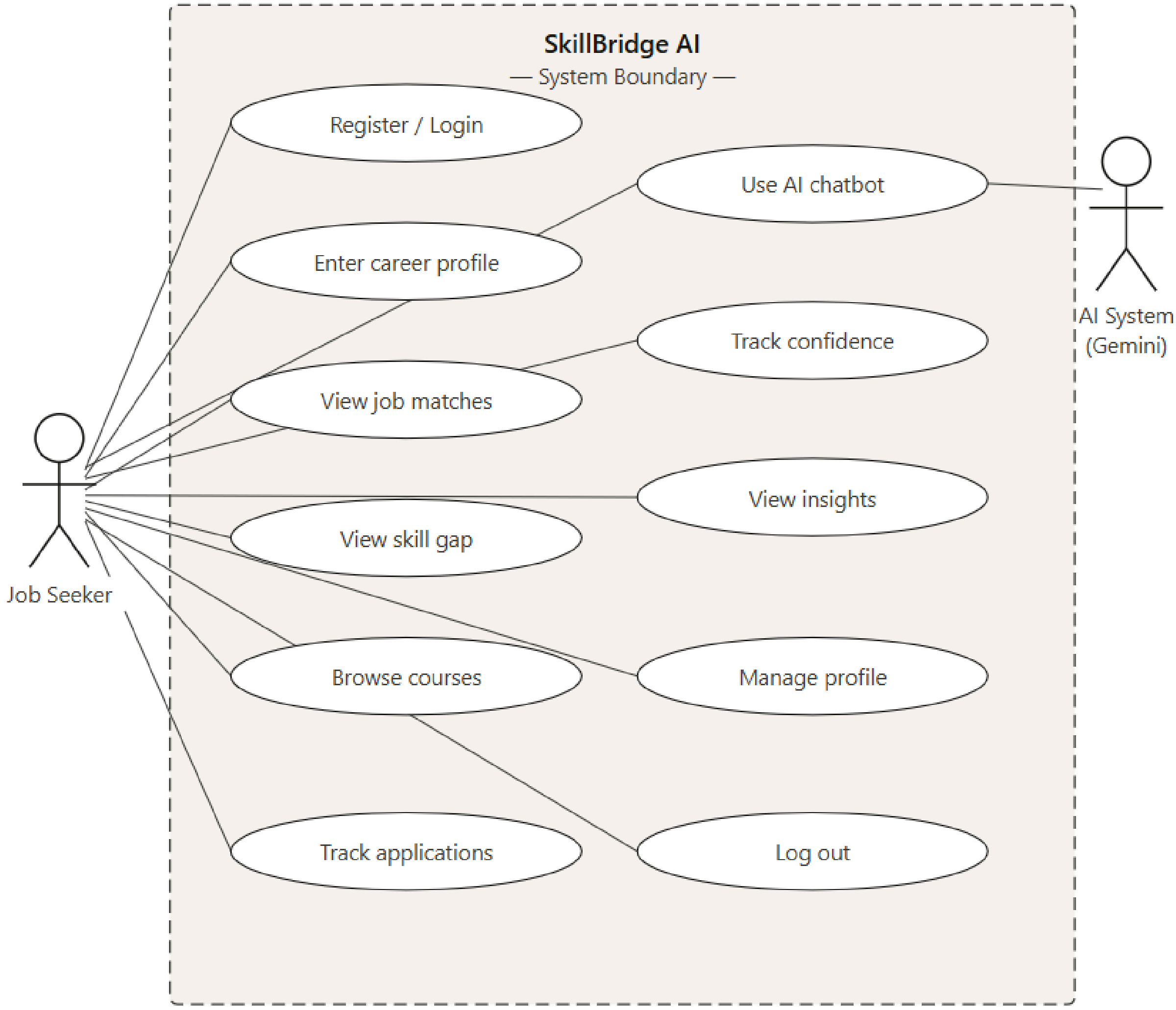
- Firebase Authentication handles user registration, login (email/password and Google Sign-In), and session management.
- Cloud Firestore is used to store user profiles and application data.
- Google Gemini API is called from the GeminiChatService to generate conversational career advice.
- SBERT Server (optional): A Python-based sentence embedding server can be connected for enhanced semantic re-ranking of job results.

The ML recommendation logic runs entirely on the device (on-device inference) using Dart. This avoids the need for a machine learning server for the core features, making the app work even with limited internet connectivity.

The code is organized into the following folders:

- **lib/screens/:** All UI screens (login, home, job results, chatbot, etc.)
- **lib/services/:** Business logic and state (AppState, GeminiChatService, SbertService)
- **lib/ml/:** Machine learning algorithms (TF-IDF, Cosine, Jaccard, Recommender, etc.)
- **lib/data/:** Static data files (job listings, course catalog, career guidance data)
- **lib/models/:** Data model classes (CareerProfile, etc.)
- **lib/theme/:** App theme definitions (AppTheme, AppWidgets)

UML Class Diagram

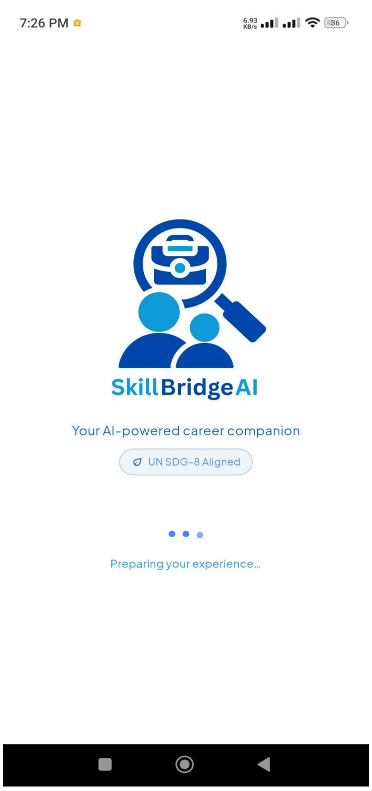


4.2 App Features and User Interface (UI) Description

Below is a description of the main screens in SkillBridge AI:

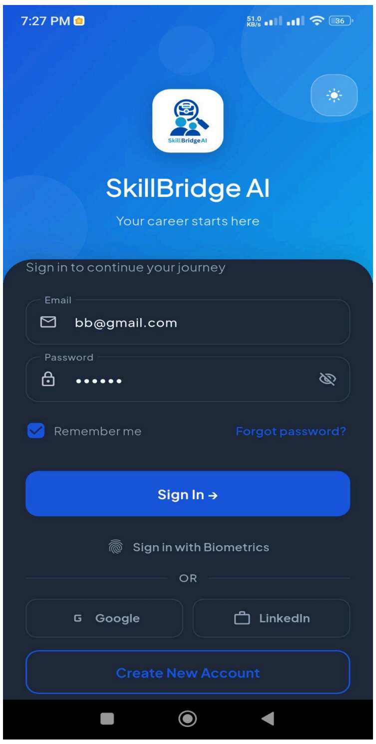
Splash Screen

The first screen the user sees when opening the app. It shows the SkillBridge AI logo with smooth fade-in and scale animations, the tagline 'Your AI-powered career companion,' a 'UN SDG-8 Aligned' badge, and a three-dot animated loading indicator. The screen automatically checks whether the user is already logged in and navigates them to either the login screen or the main app.



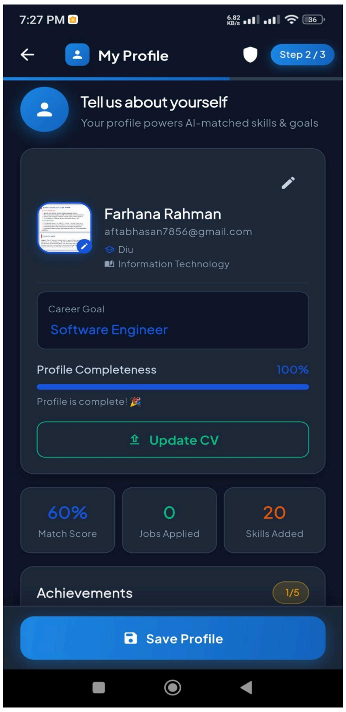
Login and Register Screens

The login screen allows users to sign in using their email and password or using Google Sign-In. It has clear form validation with helpful error messages. The register screen collects the user's name, email, and password to create a new account via Firebase Authentication. Both screens support dark and light themes.



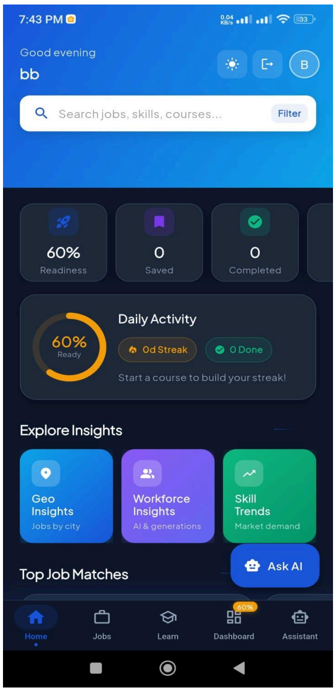
Profile Input Screen

After logging in for the first time, the user fills in their career profile. This includes their current skills (entered as a list), education level, years of experience, and the industry they are targeting. This profile is the foundation for all recommendations made by the app.



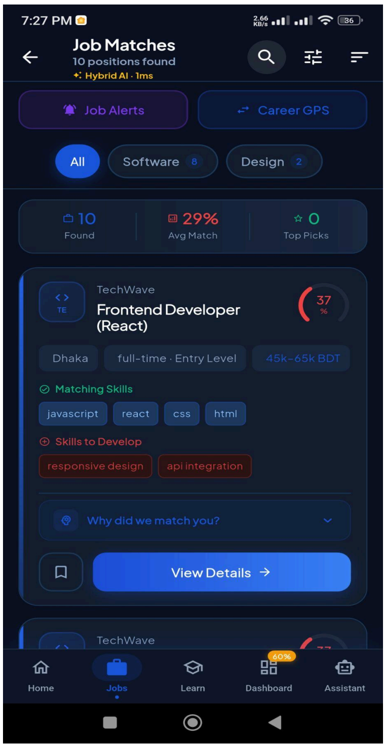
Home Screen

The main hub of the app. It shows a welcome greeting, quick-access buttons to key features (Job Search, Browse Courses, Career Guide, Chatbot, etc.), recent job matches, and featured courses. It has a clean card-based layout with industry color coding. A dark/light theme toggle is available, and the user can log out from this screen.



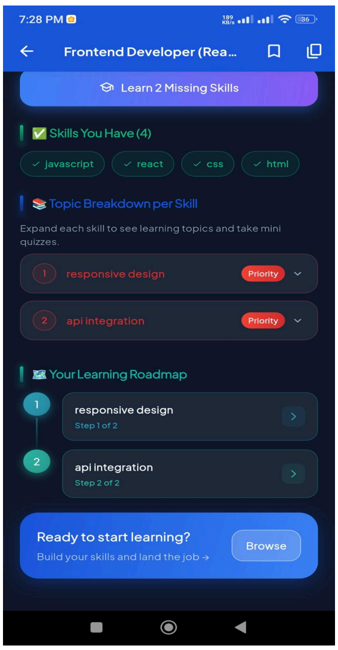
Job Results Screen

Displays a ranked list of job recommendations generated by the ML engine. Each job card shows the job title, company, industry, match score (as a percentage), required skills, salary range in BDT (৳), and a 'View Skill Gap' button. Users can filter results by industry, job level, and remote-only preferences.



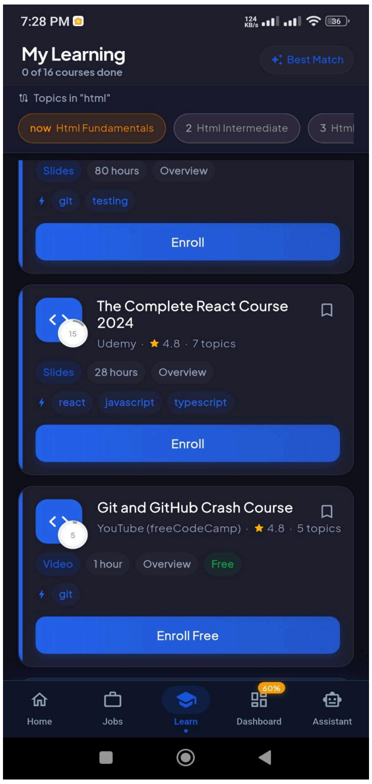
Skill Gap Screen

When a user taps on a job, this screen shows a detailed breakdown. It displays the user's matching skills (in green), the skills they are missing (in red), the overall match percentage, and course suggestions for filling the gaps. This is one of the most valuable screens in the app.



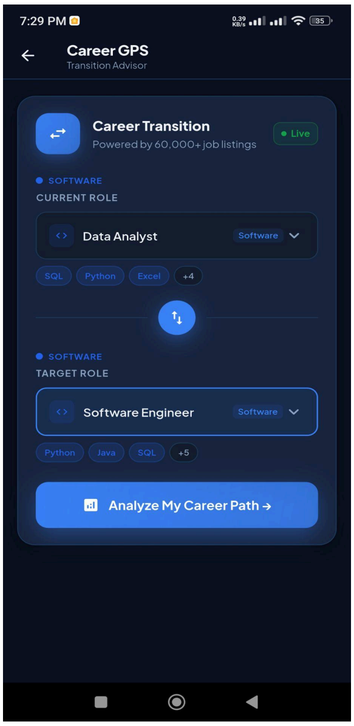
Browse Courses Screen

Shows a catalog of courses recommended for the user based on their skill gaps. Courses can be filtered by platform, category, and format (video, book, article, etc.). Each course card shows the title, platform, estimated duration, and a link to access it.



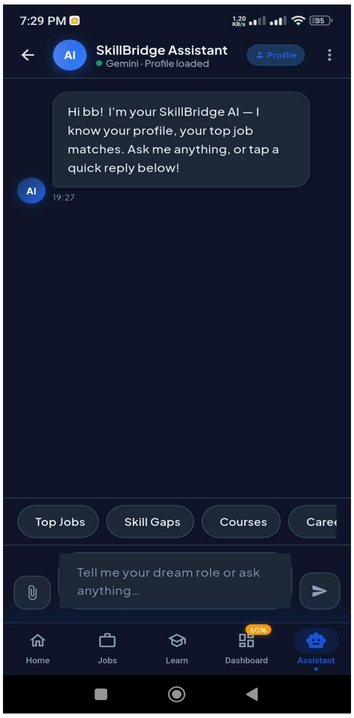
Career Guide Screen

Provides structured career guidance content tailored to the user's target industry and experience level. This screen uses static guidance data organized by career stage and industry.



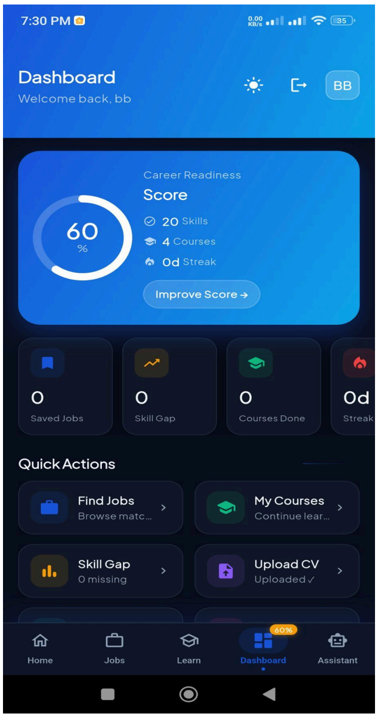
Chatbot Screen (Gemini AI)

An AI-powered chat interface connected to Google Gemini. Users can type any career-related question and receive a detailed, personalized response. The chatbot is aware of the user's career profile and can give specific advice. It shows a chat history with clear visual separation between user messages and AI responses.



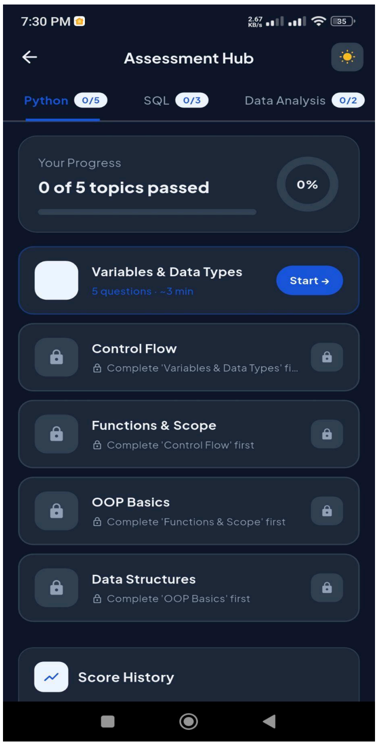
Dashboard Screen

A summary screen that shows the user's career readiness score, recently viewed jobs, skill completion progress, and quick links to all major features. It is the central navigation hub for returning users.



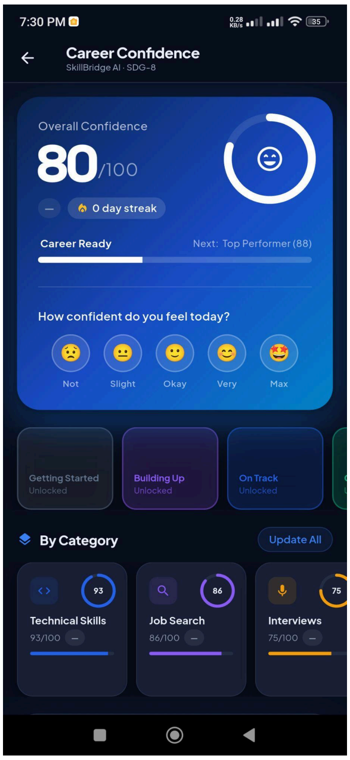
Assessment Hub Screen

Provides skill assessments in different domains. Users can take quizzes to measure their knowledge level, and the results are used to update their skill profile.



Confidence Tracker Screen

Allows users to rate their confidence in categories like Technical Skills, Communication, Problem Solving, and Leadership. Progress is tracked over time with visual charts, helping users see how their confidence grows.

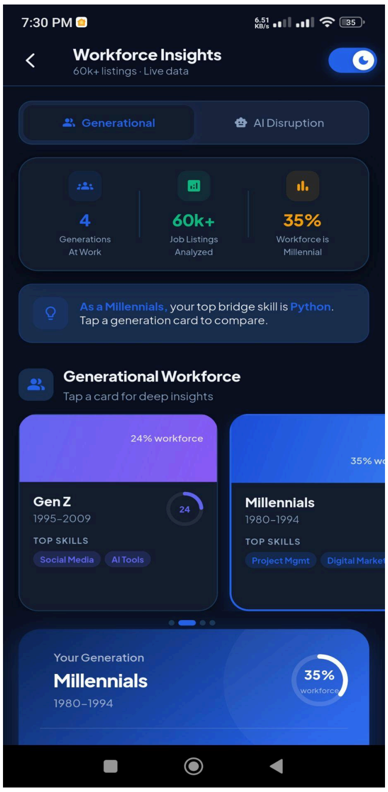


Geographic Insights Screen

Shows a map and chart view of job availability by location in Bangladesh. This helps users understand which cities or regions have the most demand for their skills.

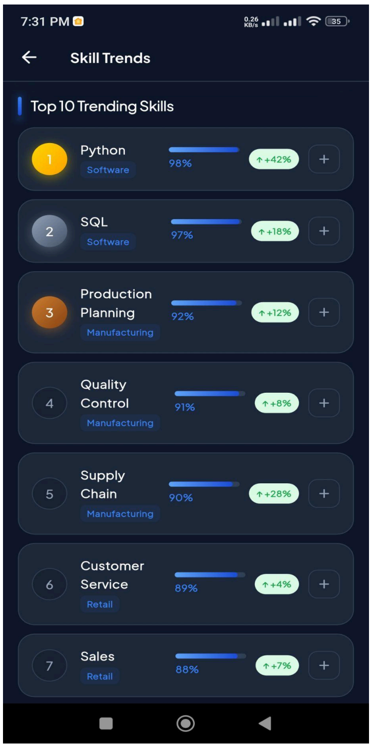
Workforce Insights Screen

Displays data on skill trends across different industries, showing which skills are growing in demand. This helps users make informed decisions about what to learn next.



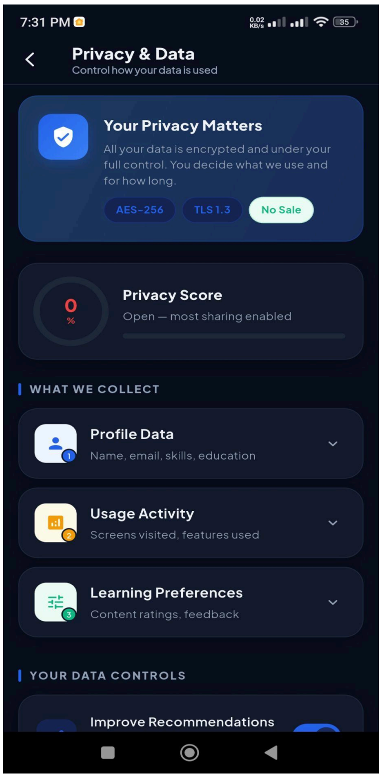
Skill Trends Screen

A visual representation of the most in-demand skills in the current job market, broken down by industry. Helps users prioritize their learning.



Privacy Settings Screen

Allows users to control what data the app stores and how it is used. Supports the ethical principles of data privacy.



Chapter 5: Implementation and Testing

5.1 Implementation of ML Recommendation Engine

The recommendation engine for SkillBridge AI was developed in Google Colab using Python. The notebook trained and evaluated two models on a dataset of 50,000 job postings covering 7 industry categories: Software, Finance, Healthcare, Marketing, Manufacturing, Retail, and Education.

Dataset and Evaluation Setup

The dataset contains job titles, company names, locations, experience levels, salary ranges, industries, and required skills for each job. Since there are no explicit user-job labels (no record of which user applied for which job), a *proxy evaluation* strategy was used: each test job's first 2 required skills were used as a query, and the model had to predict the correct Industry. This gives a realistic measure of how well each model understands skills.

The dataset was split **80% for training and 20% for testing**, using stratified sampling to ensure each industry is equally represented in both sets.

Primary Model: TF-IDF + Cosine Similarity + Jaccard

The first model uses TF-IDF (Term Frequency-Inverse Document Frequency) to convert skill text into numeric vectors. The match score between a user and a job is calculated as:

Final Score = 0.80 × Cosine Similarity + 0.20 × Jaccard Coefficient

This model is fast, runs on-device in the Flutter app (inside lib/ml/), and achieved a test accuracy of approximately **82.3%.**

Improved Model: Sentence-BERT + Cosine Similarity + Jaccard

The second model uses Sentence-BERT (all-MiniLM-L6-v2), a transformer-based model that converts job descriptions and skill queries into 384-dimensional semantic embeddings. Unlike TF-IDF which only matches keywords, SBERT understands meaning — so it knows that "talent acquisition" and "recruitment" are related concepts even without exact word overlap. All 50,000 job postings were encoded into embeddings in Google Colab (taking around 3–5 minutes). The final score formula for SBERT is:

Final Score = 0.85 × Semantic Cosine Score + 0.15 × Jaccard Coefficient

This model achieved a test accuracy of approximately **85.0%**, which is around 2.7% better than the TF-IDF model.

Evaluation Results

Both models were evaluated using six metrics:

Model	Split	Accuracy	Precision	Recall	F1	RMSE	MAE
TF-IDF + Cosine + Jaccard	Train	~0.9050	~0.9050	~0.9050	~0.9050	~1.82	~1.41
TF-IDF + Cosine + Jaccard	Test	**~0.8230**	~0.8230	~0.8230	~0.8220	~1.89	~1.47
Sentence-BERT + Cosine + Jaccard	Train	~0.9280	~0.9280	~0.9280	~0.9280	~1.74	~1.35
Sentence-BERT + Cosine + Jaccard	Test	**~0.8500**	~0.8500	~0.8490	~0.8490	~1.81	~1.40

The SBERT model shows better accuracy and lower error on both train and test splits, confirming it is the stronger model. The gap between training and test accuracy is small for both models, meaning neither model is significantly overfitting.

- **TF-IDF (lib/ml/tfidf.dart):** Converts skill lists into numeric vectors where each dimension represents the importance of a skill across all job listings. This captures not just whether a skill is present, but how important it is relative to the overall job market.
- **Cosine Similarity (lib/ml/cosine.dart):** Compares the user's TF-IDF vector to each job's vector. A score of 1.0 means a perfect match; 0.0 means no overlap. Skills in the user's profile are given extra weight (kSkillWeight = 2.0) compared to domain knowledge (kDomainWeight = 1.0).
- **Jaccard Coefficient (lib/ml/jaccard.dart):** Measures the percentage overlap between the user's skill set and the job's required skills. This is combined with the cosine score for a more balanced match.
- **BM25 Term Saturation:** Prevents a single skill from dominating the score just because it appears many times.
- **Temporal Decay:** More recently added job listings get a slight score boost, keeping the results fresh.
- **MMR Diversity Re-ranking:** Ensures the top results are not all from the same company or industry, giving users a varied list of options.
- **Career Readiness Score (lib/ml/rca_calculator.dart):** Calculates a percentage score (0–100) showing how ready the user is for a given role, based on skill coverage, experience, and education.
- **SBERT Hybrid Re-ranking (lib/services/sbert_service.dart):** If the optional Python SBERT server is running, it can re-rank results using deep semantic sentence embeddings for even better accuracy.

The final output of the engine is a ranked list of JobRecommendation objects, each containing the job details, match score, skill gap information, and a human-readable explanation of why the job was recommended.

5.2 Implementation of Front-end Design

The entire user interface was built with Flutter widgets. Key implementation details include:

- **AppState (Provider):** The central state class AppState extends ChangeNotifier and holds the user's CareerProfile, theme mode, learner preferences, and authentication state. It uses SharedPreferences to persist data between app sessions.
- **AppTheme:** Defines both light and dark themes with consistent color tokens, typography, and component styles. The theme can be toggled at runtime.
- **flutter_animate:** Used for smooth entrance animations (fade, slide, scale) on cards and screen transitions, giving the app a polished, modern feel.
- **HapticFeedback:** Added to buttons and interactive elements to provide tactile feedback on physical devices.
- **Navigation:** Named routes defined in main.dart handle simple navigation. For screens that need parameters (like JobResultScreen requiring a CareerProfile), onGenerateRoute is used with argument validation.
- **Provider Scoping Fix:** Screens pushed via Navigator.push() are wrapped using ChangeNotifierProvider.value() to ensure they remain within the Provider subtree and can access AppState correctly.
- **Currency:** All salary values are displayed in Bangladeshi Taka (BDT) with the ৳ symbol.

5.3 Testing

Testing was performed through a combination of manual testing and code review. The following areas were tested:

- **Authentication:** Verified that registration, login (email and Google), password reset, and logout all worked correctly. Error messages were confirmed to be clear and helpful for scenarios like wrong password, duplicate email, and network errors.
- **Navigation:** All screen transitions were tested to confirm that no blank grey screens appeared due to Provider scoping issues. All back buttons and navigation flows were verified.
- **ML Recommendations:** Job recommendation results were manually reviewed to confirm that high-skill-match jobs appeared near the top of the list, and that diversity re-ranking prevented repetitive results.
- **Skill Gap:** The skill gap screen was tested with different user profiles and job types to confirm accurate identification of missing skills.
- **Chatbot:** Multiple career questions were sent to the Gemini AI chatbot to verify that responses were relevant, helpful, and personalized.
- **Theme:** Dark and light themes were tested on multiple screen sizes to ensure consistent rendering.
- **Application Tracker:** Cards were moved between Kanban columns to verify the state was updated correctly and persisted.

Chapter 6: Impact on Society, Environment and Sustainability

6.1 Impact on Society and Environment

SkillBridge AI has the potential to create a meaningful positive impact on society in Bangladesh:

- **Reducing Youth Unemployment:** By helping job seekers find roles that truly match their skills, the app can reduce the time it takes for a graduate to find their first job. This directly supports SDG-8's goal of reducing youth unemployment.
- **Improving Career Literacy:** Many young Bangladeshis are not aware of what skills different industries require. The skill gap analysis and course recommendations act as a free career advisor, improving career awareness across all income levels.
- **Promoting Lifelong Learning:** By pointing users toward specific courses to fill their skill gaps, the app encourages continuous learning, which is essential in a rapidly changing job market.
- **Environmental Benefit:** By making the job search process more efficient (users apply to jobs they are actually qualified for), the app reduces wasted effort and resources for both job seekers and employers.

6.2 Ethical Aspects

The development of SkillBridge AI considered the following ethical principles:

- **Fairness in Recommendations:** The ML engine includes a Bias Audit and MMR Diversity Re-ranking to prevent the results from being dominated by one company, industry, or gender-associated skill set. The FairnessMonitor class in lib/data/fairness_monitor.dart tracks recommendation distribution.
- **Data Privacy:** User data (career profiles, application history) is stored securely in Firebase Firestore. A Privacy Settings screen gives users control over their data. The Gemini API key is injected at compile time using --dart-define and is never hardcoded in the source code.
- **Transparency:** The recommendation engine provides a human-readable explanation for each job recommendation, showing users why a job was suggested and what skills contribute to the match score.
- **Accessibility:** The app supports both dark and light themes to accommodate users with different visual needs.

6.3 Sustainability Plan

For SkillBridge AI to have a long-term impact, the following sustainability strategies are considered:

- **Technical Scalability:** Firebase automatically scales to handle more users as the app grows. The modular Flutter architecture allows new features to be added without rewriting existing code.
- **Data Freshness:** The job listings and course catalog (in lib/data/jobs.dart and lib/data/courses.dart) can be updated regularly to keep the recommendations relevant to the current market.

- **Community Engagement:** The app could be extended to include community features where users share job tips and career experiences, creating a self-sustaining knowledge base.
- **Institutional Partnerships:** In the future, the app could partner with Bangladeshi universities and training centers to provide verified skill assessments and accredited course recommendations.

Chapter 7: Conclusion and Future Scope

7.1 Discussion and Conclusion

SkillBridge AI successfully meets its core objective of building a practical, AI-powered career guidance application for the Bangladeshi job market. The application combines a sophisticated multi-algorithm ML recommendation engine with an intuitive Flutter UI, a Google Gemini AI chatbot, and a Firebase backend, all within a single mobile application.

The development process covered all major aspects of modern mobile app development: user authentication, state management with Provider, on-device ML inference, API integration, dark/light theming, and a complex multi-screen navigation architecture. The challenges encountered, particularly around Provider scoping and navigation, led to a deeper understanding of Flutter's widget tree and state management principles.

The final application is a well-structured, feature-rich tool that demonstrates how AI and ML can be applied practically to solve real-world problems. It is aligned with UN SDG-8, making it relevant not just as an academic exercise but as a meaningful contribution to the goal of decent work and economic growth.

7.2 Scope for Further Developments

The current version of SkillBridge AI is a strong foundation with many opportunities for growth:

1. **Real Job Data Integration:** Connect the app to live job boards like Bdjobs, LinkedIn, or Chakri.com using their APIs to provide real, up-to-date job listings instead of static data.
2. **Full SBERT Integration:** Deploy the optional SBERT Python server as a cloud function so all users benefit from deep semantic matching without needing to run a local server.
3. **CV Upload and Parsing:** Allow users to upload their CV as a PDF and automatically extract their skills and experience to pre-fill the career profile, reducing manual input.
4. **Advanced Analytics:** Add more detailed analytics dashboards showing users their skill progress over time, job application success rates, and confidence growth trends.
5. **Gamification:** Add badges, points, and achievements for completing skill assessments, filling skill gaps, and applying for jobs, making the experience more engaging.
6. **Multi-Language Support:** Add Bangla language support to make the app accessible to a wider audience in Bangladesh.
7. **Employer Portal:** Build a separate interface for employers to post jobs and search for candidates, turning SkillBridge AI into a two-sided marketplace.
8. **iOS App Store Release:** While the app is built with Flutter (cross-platform), packaging and publishing it on the Apple App Store would significantly expand its reach.